# Course Module:Reinforcement Learning

Latreche Sara

## Contents

**1 Introduction** **3**

**2 Historical Background** **5**

**3 Theoretical Foundations** **6**

3.1 Exploration vs. Exploitation . . . . . 6

3.2 Trade-off and $\epsilon$ Decay . . . . . 7

3.3 Hands-on Lab: Snake Game and Exploration vs. Exploitation . . . . . 8

**4 Building Your Environment** **9**

**5 Markov Decision Process (MDP)** **11**

5.1 Historical Background of Markov Chains . . . . . 11

5.2 Bellman Equation for MDPs . . . . . 13

5.3 Hands-on Lab: Tic-Tac-Toe and Bellman Values . . . . . 14

5.4 Supplementary Video: Q-Learning in Tic-Tac-Toe . . . . . 14

**6 4. Core Algorithms** **16**

6.1 Temporal Difference (TD) Learning . . . . . 16

6.2 Q-Learning . . . . . 19

6.3 SARSA . . . . . 19

6.4 Comparison of Q-Learning vs SARSA . . . . . 20

6.5 Deep Q-Network (DQN) . . . . . 20

**7 Hands-On Labs** **22**

**8 Applications & Capstone Projects** **22**

# 1 Introduction

As a subfield of machine learning, reinforcement learning (RL) has emerged as a powerful paradigm for sequential decision making. Unlike supervised learning methods that rely on one-shot and exhaustive labels, RL learns policies through trial-and-error interactions with the environment, guided only by evaluative and often delayed feedback. This distinctive capability allows RL to address problems where optimal actions must be discovered dynamically and improved over time.

Over the last decade, RL has gained increasing attention from both academia and industry, driven by advances in deep learning, scalable computation, and the availability of massive multimodal data. Its impact spans a wide range of domains, including autonomous driving, quantitative finance, and healthcare. In the context of self-driving cars, deep reinforcement learning has been widely studied for motion control, navigation, and behavior planning [5]. Startups such as Wayve.ai, Zoox, Cruise, and Pony.ai exemplify the translation of these methods into real-world innovation. Similarly, RL techniques have been applied in financial markets, enabling advances in portfolio optimization, trading strategies, and risk management [6]. In healthcare, RL is proving to be a transformative tool for dynamic treatment regimes, adaptive clinical trials, automated diagnosis, and drug discovery [7]. These applications highlight the versatility of RL in addressing problems characterized by uncertainty, sequential decision processes, and the need for long-term optimization.

Despite this progress, RL remains a rapidly evolving field with open challenges in sample efficiency, interpretability, and safe deployment in high-stakes domains. The purpose of this course is to provide both a conceptual and practical foundation of reinforcement learning, while motivating learners through exposure to cutting-edge applications that are shaping the future of AI.




Figure 1: Illustration of reinforcement learning applications.

**Definition**

**Reinforcement Learning (RL)** is the science of decision-making in **uncertain and dynamic environments**.
Have you ever wondered how AI makes decisions in unpredictable situations? That's where RL comes in. It is used in:

- Robotics and autonomous vehicles
- Playing complex games like Chess or AlphaGo
- Real-world control systems where rules are not fixed

The agent learns **optimal behavior** through continuous interaction with its environment:

- **Agent:** the decision-maker
- **Environment:** the world in which the agent operates
- **Policy:** the strategy the agent follows to take actions
- **Reward signal:** feedback (positive or negative) from the environment

Unlike other ML approaches that require fixed datasets, RL solves problems as a **whole process of interaction**, making it ideal for **dynamic, uncertain environments**.

**Active Recall – Definition of RL**

Q: What makes reinforcement learning different from supervised learning?
A: RL does not need labeled data — instead, it learns through trial-and-error interaction with the environment using rewards and penalties.

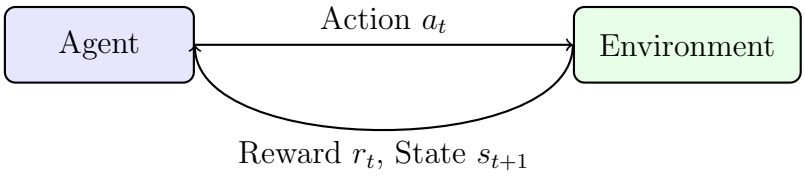



Figure 2: Core components of Reinforcement Learning: the agent interacts with the environment by taking actions, and receives new states and rewards as feedback.

# 2 Historical Background

**Example**

Reinforcement Learning has evolved over decades, shaped by key milestones:

- **1950s:** Richard Bellman introduces *Dynamic Programming*, laying the foundation for sequential decision-making.
- **1960s:** Arthur Samuel builds a self-learning *Checkers program*, one of the first demonstrations of machine learning.
- **1980s:** Richard Sutton develops *Temporal-Difference (TD) learning*, bridging Monte Carlo and Dynamic Programming.
- **1992:** Chris Watkins proposes *Q-learning*, a model-free method that becomes a cornerstone of modern RL.
- **2013–2016:** DeepMind demonstrates RL at scale with *DQN on Atari games* and the landmark success of *AlphaGo*.

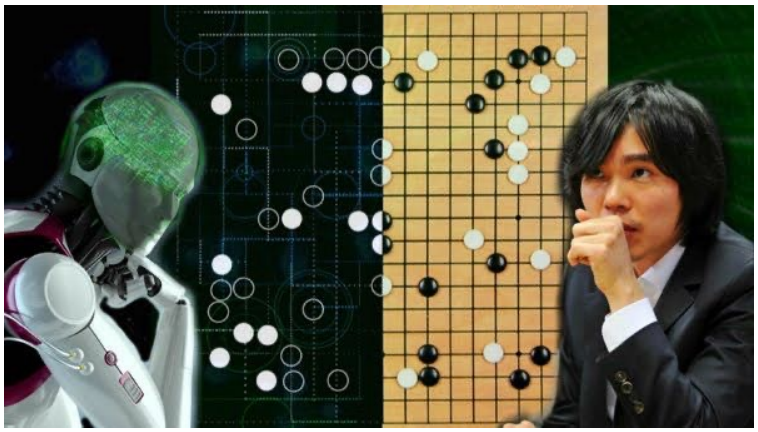


Figure 3: AlphaGo defeating world champion Lee Sedol (2016) — a landmark achievement for Reinforcement Learning.

# 3 Theoretical Foundations

## 3.1 Exploration vs. Exploitation

**Definition**

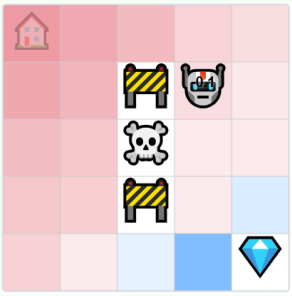
**Exploitation:** Pick the best-known action that maximizes reward based on current knowledge.

**Exploration:** Try new actions to discover potentially better rewards.

**Example**

**Grid-World Diamond Game Example:** Consider an RL agent navigating a grid-world where the goal is to reach the diamond:

- **Exploitation:** The agent always takes the known shortest path directly to the diamond, maximizing immediate reward.
- **Exploration:** The agent sometimes tries alternative paths, which may reveal safer routes, avoid penalties, or even uncover faster ways to reach the diamond.



**Note**

A common strategy to balance exploration and exploitation is the **$\epsilon$-greedy algorithm**:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|}, & \text{if } a = \arg\max_{a'} Q(s, a') \\ \frac{\epsilon}{|A|}, & \text{otherwise} \end{cases}$$

where:

- $\epsilon$: probability of exploring a random action
- $1 - \epsilon$: probability of exploiting the best-known action
- $|A|$: number of possible actions

## 3.2 Trade-off and $\epsilon$ Decay

**Note**

To make learning efficient, $\epsilon$ is often decreased over time:

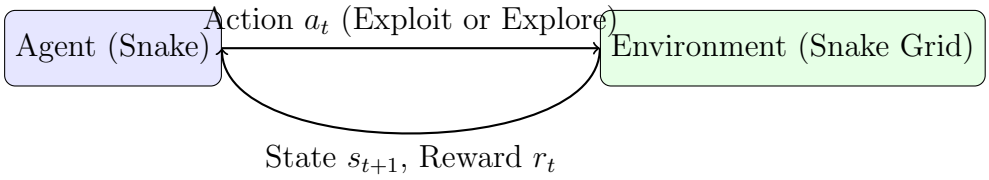
$$\epsilon_t = \epsilon_0 \cdot \text{decay}^t$$

where $\epsilon_0$ is the initial exploration rate (e.g., 1.0) and decay is slightly less than 1 (e.g., 0.99).
**Why?**

- Early: high exploration ensures the agent discovers multiple strategies.
- Later: lower $\epsilon$ favors exploitation, allowing the agent to maximize reward using what it has learned.

**Active Recall – Exploration vs. Exploitation**

Q: What happens if an agent only exploits? A: It may miss discovering better strategies.
Q: What happens if an agent only explores? A: It wastes time and never leverages what it has already learned.
Q: Why decrease $\epsilon$ over time? A: To shift the balance from exploration to exploitation as the agent gains experience.




$\epsilon$ controls exploration vs exploitation
High $\epsilon$ = more exploration
Low $\epsilon$ = more exploitation

Figure 4: The agent-environment loop with $\epsilon$-greedy trade-off.

**Note**

**Other Exploration Strategies:** While $\epsilon$-greedy is the most widely used approach, several alternatives exist:

- **Boltzmann (Softmax) Exploration:** actions chosen with probability proportional to their estimated value.
- **Heuristic-based Exploration:** use domain knowledge to guide exploration.
- **Bayesian Exploration:** model uncertainty over $Q$-values and sample accordingly.
- **Count-Based Exploration:** favor actions/states that have been visited less often.
- **Intrinsic Motivation:** encourage novelty by rewarding exploration of surprising states.

## 3.3 Hands-on Lab: Snake Game and Exploration vs. Exploitation

**Hands-on Lab**

In this lab, you will observe how an RL agent learns to play the Snake game while balancing exploration and exploitation.
**Objective:** Understand how $\epsilon$-greedy policy influences the agent's behavior.

1. Run the Snake game agent with a high $\epsilon$ (e.g., 1.0). Observe: the agent explores many paths randomly.
2. Gradually decrease $\epsilon$ over episodes (e.g., $\epsilon_t = \epsilon_0 \cdot 0.99^t$). Observe: the agent starts exploiting better paths it has learned.
3. Answer the following questions:
   (a) How does the snake's score change as $\epsilon$ decreases?
   (b) What happens if $\epsilon$ is fixed at 1.0? What if it is fixed at 0.0?
   (c) How does exploration help the agent avoid collisions and discover better paths to food?

**Optional Extension:** Try modifying the reward structure (e.g., make eating an apple give +20 instead of +10, or hitting walls give -200) and observe how the agent's behavior changes.

**Note**

**From Gym to Gymnasium**

- **OpenAI Gym** was originally created by OpenAI (2016) as the standard toolkit for Reinforcement Learning research and benchmarking.
- In 2022, OpenAI **archived Gym** and stopped maintaining it.
- The **Farama Foundation** took over and released **Gymnasium**, a modern, maintained, drop-in replacement that fixes compatibility issues (e.g., with NumPy 2.0) and adds new features.
- Today, Gymnasium is the recommended toolkit to build and test RL environments.

# 4 Building Your Environment

In reinforcement learning, the **environment** is the world in which the agent acts, receives feedback, and learns.

## Types of Environments

- **Deterministic vs. Stochastic:** Deterministic environments always give the same outcome for the same action, while stochastic environments include randomness.
- **Episodic vs. Continuing:** Episodic tasks end after a sequence of steps (e.g., board games). Continuing tasks run without a fixed endpoint (e.g., traffic control).
- **Discrete vs. Continuous:** States and actions can be finite and countable (e.g., moves in chess), or continuous (e.g., steering angles in autonomous driving).

**RL Platforms**

Reinforcement learning **platforms** provide standardized environments to design, simulate, and evaluate agents. They make it possible to benchmark algorithms and compare results across research studies. A good platform should be:

- Easy to use and well-documented.
- Flexible enough to cover both simple and complex tasks.
- Widely adopted, so results are reproducible and comparable.

## Popular Platforms

- **OpenAI Gym**[1] — Created by *OpenAI*, a research lab co-founded by Elon Musk and Sam Altman. Gym has become the most widely used RL platform, offering standardized environments such as CartPole, MountainCar, Atari games, and robotics simulations. It is simple, well-documented, and often the first step for RL experiments.
- **PettingZoo**[2] — A companion to Gym, specialized for multi-agent environments.
- **DeepMind Control Suite**[3] — Provides physics-based continuous control benchmarks.
- **Unity ML-Agents**[4] — A powerful 3D simulation platform to build custom environments.
- **Project Malmo**[5] — Built on Minecraft by Microsoft, useful for flexible AI experimentation.

Among these, **OpenAI Gym** is the entry point for most practitioners and remains the backbone for many RL toolkits.

**Hands-on Lab: Building Environments with Gymnasium**

In this lab, you will explore how to create and interact with environments using **Gymnasium**, the modern successor to OpenAI Gym.
**Guiding Questions:**

1. How do you create a Gymnasium environment (e.g., `FrozenLake-v1`) using `gym.make()`?
2. What are the **observation space** and **action space** of the environment? How can you inspect them?
3. What happens when you call `env.reset()` and `env.step(action)`?
4. How does the environment communicate information back to the agent (state, reward, termination, truncation, info)?
5. Why is Gymnasium useful for prototyping RL algorithms compared to coding environments from scratch?

**Next Step:** Open the provided Jupyter notebook and experiment with the `FrozenLake-v1` environment. Try printing the state transitions and observing how Gymnasium handles stochastic outcomes.

[1] https://www.gymlibrary.dev
[2] https://www.pettingzoo.farama.org
[3] https://github.com/deepmind/dm_control
[4] https://unity.com/products/machine-learning-agents
[5] https://www.microsoft.com/en-us/research/project/project-malmo

# 5 Markov Decision Process (MDP)

## 5.1 Historical Background of Markov Chains

### Nekrasov vs. Markov

At the turn of the 20th century, debates in Russia about free will and determinism spilled into mathematics. **Pafnuty Nekrasov** argued that statistical regularities—such as the balance of births and deaths over decades—proved the existence of free will or even divine order. **Andrey A. Markov** strongly disagreed. He insisted that the *law of large numbers* was a mathematical fact and said nothing about metaphysical freedom or divine purpose.

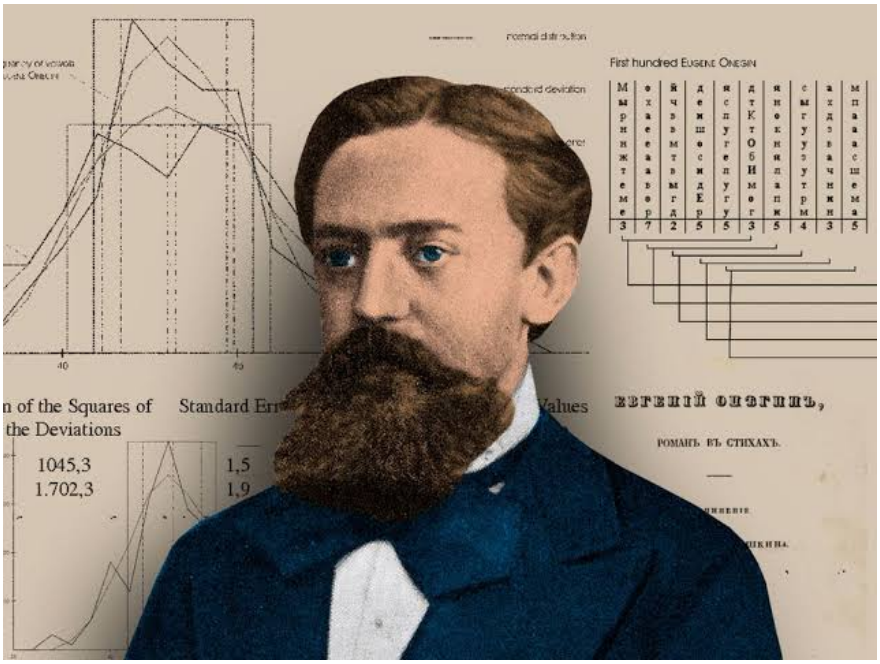



Figure 5: Andrey A. Markov (1856–1922). His study of Pushkin's poetry laid the foundation for Markov chains.

### Pushkin's Poem and the Birth of Markov Chains

To challenge Nekrasov, Markov turned to literature. He analyzed Alexander Pushkin's poem *Eugene Onegin*, counting how often vowels followed consonants and vice versa. He discovered that the probability of the next letter depended mostly on the current one, not on the whole history of the text.

This gave rise to the **Markov property**: the future state of a process depends only on the present state, not on the entire past. Thus, the law of large numbers could apply even to dependent events.

## From Poetry to Physics

Decades later, the idea that randomness could reveal structure reappeared in physics. **Stanislaw Ulam**, recovering from illness, amused himself by calculating the chances of winning solitaire using random trials. This led him and **John von Neumann** to develop the **Monte Carlo method** at Los Alamos during the Manhattan Project. Random sampling made it possible to estimate probabilities in complex chain reactions.

## From Atoms to Autocomplete

The conceptual leap from Pushkin's poetry to nuclear physics also shaped modern computing. Early text-prediction systems and search engines used Markov-like models (*n-grams*, hidden Markov models) to guess the next word or the next click. Today's neural networks are far more sophisticated, but the heart of the idea is the same:

The present is enough to predict the immediate future.

## Study questions

1. **Letter experiment:** Take a short English paragraph. Count how often a vowel follows a consonant. Do you see patterns?
2. **Discussion:** Why does the law of large numbers not prove free will? Defend Markov's view in a short paragraph.
3. **Modern link:** Find one modern use of sequence models (e.g., speech recognition, autocomplete). Explain in two sentences how it works.

**Definition**

A **Markov Decision Process (MDP)** is a framework for modeling sequential decision-making problems, defined by the tuple $(S, A, P, R, \gamma)$:

- $S$: Set of all possible states
- $A$: Set of possible actions
- $P(s'|s, a)$: Transition probability to next state $s'$ given state $s$ and action $a$
- $R(s, a)$: Reward received after taking action $a$ in state $s$
- $\gamma$: Discount factor for future rewards

**Example**

**Tic-Tac-Toe Example:**

- **State** $S$**:** Current board configuration (3×3 grid)
- **Action** $A$**:** Place your mark (X or O) in an empty cell
- **Transition** $P$**:** The chosen action results in a new board
- **Reward** $R$**:** +1 for a win, 0 for a draw, -1 for a loss

The agent learns which moves maximize long-term reward through trial and error.

**Note**

**Markov Property:** The next board configuration depends only on the current state and action, not on the previous history. This is what makes Tic-Tac-Toe an MDP.

## 5.2 Bellman Equation for MDPs

The value of a state $s$ under a policy $\pi$ is given by the **Bellman expectation equation**:

$$\boxed{V_\pi(s) = \sum_{a \in A} \pi(a \mid s) \Big[ R(s,a) + \gamma \sum_{s' \in S} P(s' \mid s,a) V_\pi(s') \Big]}$$

where:

- $V_\pi(s)$: value of state $s$ under policy $\pi$
- $\pi(a \mid s)$: probability of taking action $a$ in state $s$
- $R(s,a)$: reward after taking action $a$ in state $s$
- $P(s' \mid s,a)$: probability of transitioning to $s'$ from $s$ with action $a$
- $\gamma$: discount factor for future rewards

The **optimal value function** selects the action that maximizes expected return:

$$\boxed{V^*(s) = \max_{a \in A} \Big[ R(s,a) + \gamma \sum_{s' \in S} P(s' \mid s,a) V^*(s') \Big]}$$

## 5.3 Hands-on Lab: Tic-Tac-Toe and Bellman Values

**Hands-on Lab**

In this lab, you will connect the numerical Bellman equation example for Tic-Tac-Toe with an actual RL agent that learns to play the game.
**Objective:** Understand how Bellman equations guide decision-making in Tic-Tac-Toe and how simulation approximates these values through repeated play.

1. Recall the numerical example: in a given board state, playing in cell 4 leads to an immediate win ($V(s) = 1$), while playing in cell 6 gives only partial value ($V(s) = 0.45$).
2. Run the Tic-Tac-Toe RL agent. Let it train until it can play against you.
3. Observe: Does the agent prefer the winning move (like action 4) over uncertain moves (like action 6)?
4. Modify the parameters and re-run training:
   - Change the learning rate $\alpha$. What happens to learning speed?
   - Change the exploration rate $\epsilon$. Does the agent play more randomly or more predictably?
5. Answer the following questions:
   (a) How does the Bellman equation explain why action 4 is better than action 6?
   (b) In the code, where do you see values being updated to reflect Bellman's principle?
   (c) Why is simulation useful when the game is more complex than Tic-Tac-Toe?

## 5.4 Supplementary Video: Q-Learning in Tic-Tac-Toe

To visualize how an agent learns to play Tic-Tac-Toe using Q-Learning, watch the following video:

**Tic-Tac-Toe and Q-Learning Demonstration on YouTube**

**Key points to focus on while watching:**

1. **State Representation:** Each board configuration is treated as a distinct state.
2. **Q-Table:** The table stores expected rewards for each state-action pair and updates as the agent plays.

3. **Exploration vs. Exploitation:** The agent balances trying new moves and using known strategies.
4. **Learning Process:** Observe how repeated gameplay refines the agent's strategy.
5. **Convergence:** Notice how the Q-values stabilize after sufficient training.

**Optional Hands-on Activity:**

- Pause the video and try predicting the agent's next move.
- Compare your strategy with the agent's Q-values.
- Discuss how the Q-Learning update rule is applied at each step.

# 6 4. Core Algorithms

**Definition**

## 6.1 Temporal Difference (TD) Learning

**Temporal Difference (TD) Learning:** A model-free reinforcement learning method that updates estimates of the value function based on **partial experience**, using the current reward and the estimated value of the next state. Unlike Monte Carlo methods, TD can learn **online**, step by step, without waiting for the episode to finish.

**Value Function vs Action-Value Function**

**Value Function** $V(s)$: Measures how good it is to be in a given state $s$ under a policy $\pi$:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\Big[\sum_{t=0}^{\infty} \gamma^t r_t \,\Big|\, s_0 = s\Big]$$

**Action-Value Function (Q-function)** $Q(s, a)$: Measures how good it is to take an action $a$ in state $s$:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\Big[\sum_{t=0}^{\infty} \gamma^t r_t \,\Big|\, s_0 = s, a_0 = a\Big]$$

**Relation:**

$$V^{\pi}(s) = \max_a Q^{\pi}(s, a)$$

**Policy:** $\pi(s)$ defines which action to take in each state. The optimal policy from the Q-table:

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

**Q-Table**

A Q-table stores expected future rewards for each state-action pair. Example for Tic-Tac-Toe:

| State | Action 1 | Action 2 | Action 3 |
|---|---|---|---|
| s1 | 0.0 | 0.0 | 0.0 |
| s2 | 0.0 | 0.0 | 0.0 |

**Note**

**Monte Carlo (MC) vs Temporal Difference (TD):**

- **MC:** Update at the end of the episode using full return $G = \sum r_t$. Inefficient for long sequences.
- **TD:** Update at each step using the next state estimate. More sample-efficient and suitable for online learning.

**TD(0) Algorithm**

TD(0) is an **on-policy** algorithm that updates the value of states based on the observed rewards and the next state.

1. Observe the current state $s$ and take an action $a$ according to the policy $\pi$.
2. Receive reward $r$ and observe the next state $s'$.
3. Update the value of the current state:

$$V(s) \leftarrow V(s) + \alpha\big(r + \gamma V(s') - V(s)\big)$$

4. Repeat until the episode ends.

**Parameters:** $\alpha$ = learning rate, $\gamma$ = discount factor
**Variants:** TD($\lambda$), SARSA (on-policy)

**Neuroscience Connection**

The TD error:

$$\delta = r + \gamma V(s') - V(s)$$

is analogous to **dopamine reward prediction error** in the brain. Dopamine signals encode the difference between expected and received rewards, guiding learning similarly to TD updates in reinforcement learning.

**Active Recall – Tricky Question – Policy Evaluation**

Suppose an agent has an incomplete Q-table. Some states have missing values. Can you still deduce the optimal action in a given state using $V(s)$? Why or why not?
**Hint:** Think about the relation $V(s) = \max_a Q(s, a)$ and missing Q-values.

**Active Recall – Tic-Tac-Toe Reasoning**

Consider the board state:

| | | |
|---|---|---|
| X | O | |
| | X | |
| O | | |

Assume the Q-table gives the following values for the empty cells:

$$Q(s, \text{top-right}) = 0.2, \quad Q(s, \text{middle-left}) = 0.4, \quad Q(s, \text{bottom-middle}) = 0.3$$

**Question:** Which action should the agent choose according to TD learning policy?
**Bonus:** How would this decision change if the discount factor $\gamma$ was very small?

**Active Recall – Neuroscience Connection**

Explain how the TD error $\delta = r + \gamma V(s') - V(s)$ can be interpreted in the brain in terms of dopamine signals. **Hint:** Consider what happens when the reward is higher or lower than expected.

## 6.2 Q-Learning

**Definition**

**Q-Learning:** A model-free, off-policy TD learning algorithm that learns the **optimal action-value function** $Q^*(s,a)$ regardless of the policy being followed. It updates Q-values using the maximum estimated reward from the next state, allowing the agent to learn the optimal policy.

**Note**

**Intuition:** - Q-Learning is a **TD learning method** because it updates estimates step by step using the next state's estimated values. - Unlike regular TD(0) for V(s), Q-Learning works with **Q(s,a)**, storing expected returns for state-action pairs. - The update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$$

means we adjust Q(s,a) toward the best possible value in the next state ($\max_{a'} Q(s',a')$), independent of the action actually taken next.

**Algorithm 1** Q-Learning

Initialize $Q(s,a)$ arbitrarily
**for** each episode **do**
    Initialize state $s$
    **while** not terminal **do**
        Choose $a$ using $\epsilon$-greedy
        Take action $a$, observe $r, s'$
        $Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
        $s \leftarrow s'$
    **end while**
**end for**

## 6.3 SARSA

**Definition**

**SARSA:** A model-free, on-policy TD control algorithm. It updates Q-values using the **action actually taken** in the next state, rather than the max:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big]$$

**Note**

**Intuition:** - SARSA is **on-policy**: it learns the Q-values of the policy the agent is actually following ($\epsilon$-greedy). - Q-Learning is **off-policy**: it learns the optimal Q-values even if the agent follows a different policy during exploration.

## 6.4 Comparison of Q-Learning vs SARSA

**Comparison of Q-Learning vs SARSA**

| Aspect | Q-Learning | SARSA |
|---|---|---|
| Policy | Off-policy | On-policy |
| Update rule | $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ | $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$ |
| Exploration effect | Learns optimal regardless of exploration | Updates depend on actual actions taken, includes exploration |
| Behavior | Risk-neutral (assumes optimal next action) | Risk-aware (follows current policy) |

## 6.5 Deep Q-Network (DQN)

**Note**

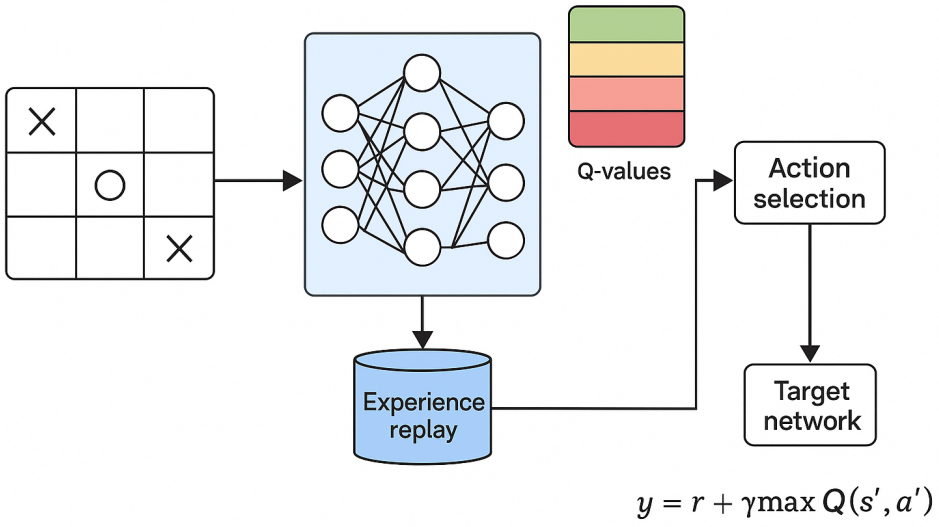
DQN uses a neural network to approximate $Q(s,a)$ instead of a lookup table.

**Intuition:** Traditional Q-Learning stores Q-values for every state-action pair, which becomes infeasible for large or continuous state spaces. A Deep Q-Network approximates the Q-function using a neural network, $Q(s,a;\theta)$, where $\theta$ are the network parameters.

Key ideas: - **Function Approximation:** Use a neural network to estimate Q-values. - **Experience Replay:** Store transitions $(s,a,r,s')$ in a memory buffer and sample mini-batches to break correlation between sequential experiences. - **Target Network:** Use a separate network to compute the target Q-values to stabilize learning.

**Algorithm 2** Deep Q-Learning (DQN)

Initialize replay memory $D$ to capacity $N$
Initialize action-value function $Q$ with random weights $\theta$
Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$
**for** each episode **do**
    Initialize state $s$
    **while** $s$ is not terminal **do**
        Choose $a$ from $s$ using policy derived from $Q$ (e.g., $\epsilon$-greedy)
        Take action $a$, observe reward $r$ and next state $s'$
        Store transition $(s, a, r, s')$ in $D$
        Sample random mini-batch of transitions $(s_j, a_j, r_j, s'_j)$ from $D$
        Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j \\ r_j + \gamma \max_{a'} \hat{Q}(s'_j, a'; \theta^-) & \text{otherwise} \end{cases}$
        Perform a gradient descent step on $(y_j - Q(s_j, a_j; \theta))^2$ with respect to $\theta$
        Periodically update target network: $\theta^- \leftarrow \theta$
        $s \leftarrow s'$
    **end while**
**end for**




**Simple Illustrative Example (Tic-Tac-Toe):** - Instead of a Q-table, represent the board state as a 9-dimensional vector (1 for X, -1 for O, 0 for empty). - Neural network inputs: board vector; outputs: Q-values for each possible action (cell 1–9). - During training: sample past experiences from memory, compute targets with the target network, and update the Q-network. - Over episodes, the network learns to predict the expected reward for each action given a board state, effectively learning an optimal policy.

# 7 Hands-On Labs

**Hands-on Lab**

**Lab 1: Multi-Armed Bandit** Implement $\epsilon$-greedy strategy to maximize reward.

**Hands-on Lab**

**Lab 2: Q-Learning in Gridworld** Train an agent to avoid cliffs using Q-learning.

**Hands-on Lab**

**Lab 3: DQN on CartPole** Use PyTorch to train an agent to balance the pole.

# 8 Applications & Capstone Projects

**Capstone Ideas**

- Train an Atari agent (DQN on Breakout).
- Optimize traffic lights with RL.
- Build a robotic arm controller in simulation.
- Apply RL to recommendation systems.

# References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

2. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292.

3. Rummery, G. A., & Niranjan, M. (1994). *On-line Q-learning using connectionist systems* (Technical Report CUED/F-INFENG/TR 166). Cambridge University Engineering Department.

4. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

5. B. Ravi Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, *Deep Reinforcement Learning for Autonomous Driving: A Survey*, arXiv:2002.00444, 2020.

6. T.-V. Pricope, *Deep Reinforcement Learning in Quantitative Algorithmic Trading: A Review*, arXiv:2106.00123, 2021.

7. C. Yu, J. Liu, and S. Nemati, *Reinforcement Learning in Healthcare: A Survey*, arXiv:1908.08796, 2019.